Computer-Assisted Formalization of Mathematics

Week 09 - Metaprogramming I: Lean under the hood

Jaume de Dios Pont

New York University

November 5, 2026

Programming Lean (in Lean)

We have written programs that manipulate arithmetic expressions.

```
Expr.eval : Expr α → (α → ℤ) → ℤ
```

Programming Lean (in Lean)

We have written programs that manipulate arithmetic expressions.

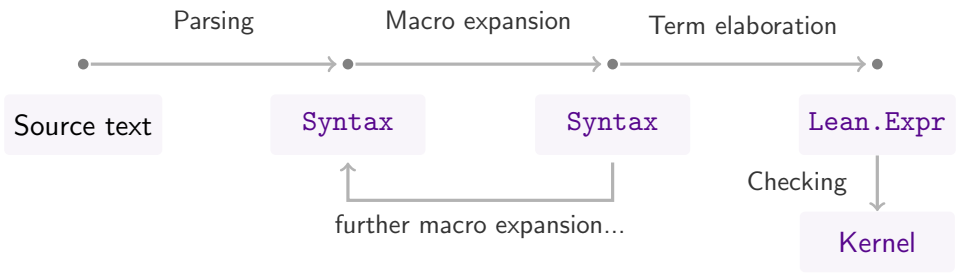
```
Expr.eval : Expr α → (α → ℤ) → ℤ
```

We can also write programs that manipulate **Lean programs and proofs**.

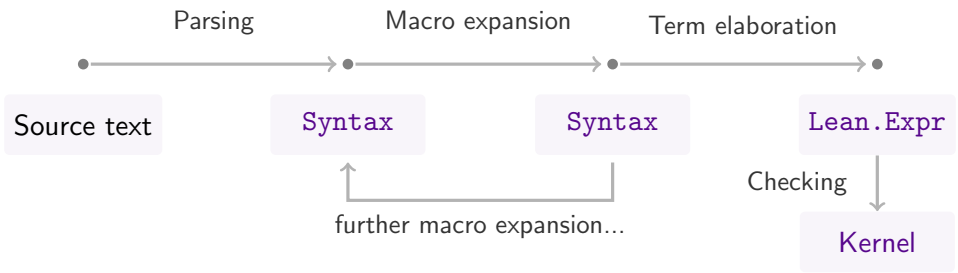
- Introduce new mathematical notation.
- Generate definitions or proof terms.
- Inspect a goal and choose a tactic.

This is **metaprogramming**. We use Lean itself to do it.

From text to a proof term



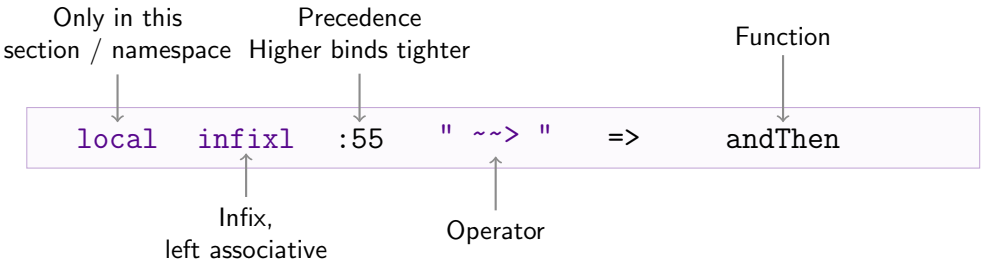
From text to a proof term



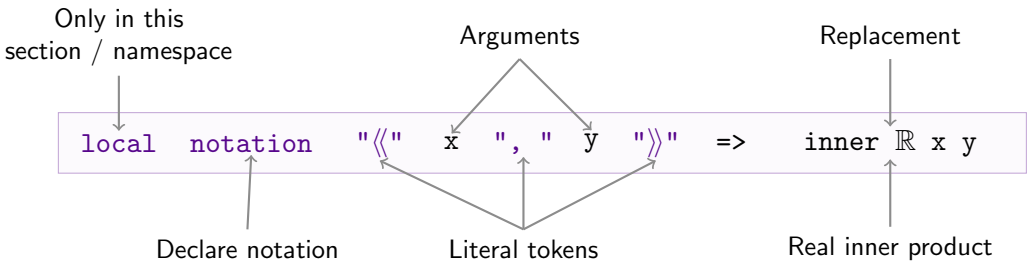
Elaboration resolves names, infers types, and fills implicit arguments.

The **kernel** checks the resulting declaration and its proof term.

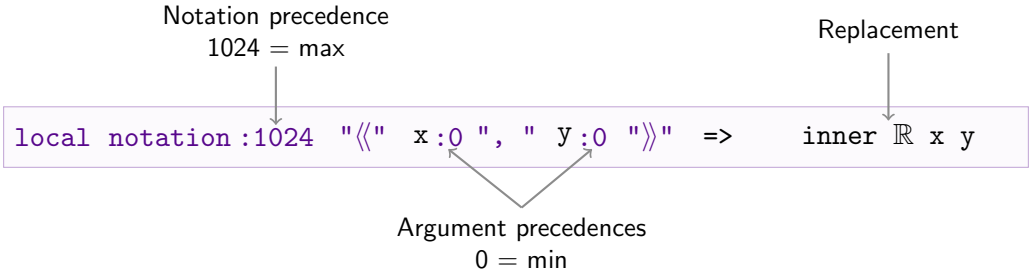
Infix


Only in this
section / namespace
Precedence
Higher binds tighter
Function
local infixl :55 " ~~> " => andThen
Infix,
left associative
Operator

Notation

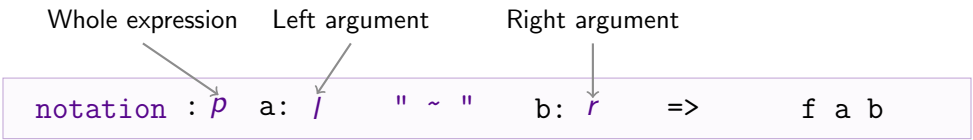

Only in this section / namespace
Arguments
Replacement
local notation "⟪" x ", " y "⟫" => inner ℝ x y
Declare notation
Literal tokens
Real inner product

Notation: precedence



The same notation, with its default precedences written explicitly.

Infix from notation



	Whole p	Left l	Right r	Grouping
`infixl:`p	p	p	$p+1$	$(a \sim b) \sim c$
`infixr:`p	p	$p+1$	p	$a \sim (b \sim c)$
`infix:`p	p	$p+1$	$p+1$	Explicit parentheses

Syntax categories

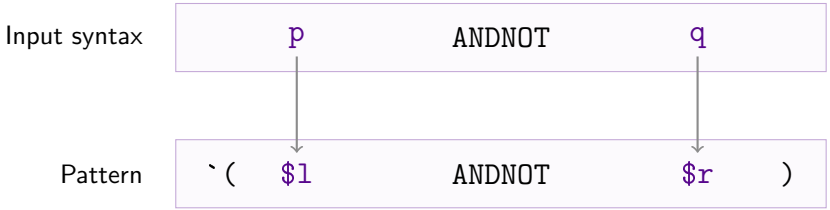
```
scoped syntax:10 term:10 " ANDNOT " term:11 : term
```

Examples of syntax categories

Result category

`term`	`1 + 1`, `Nat`, $a \wedge b$
`command`	`#check Nat`, `def n := 3`
`tactic`	`rfl`, `apply And.intro`

Matching syntax



A quotation can be a pattern.

`$l` and `$r` capture syntax pieces.

Syntax, simplified

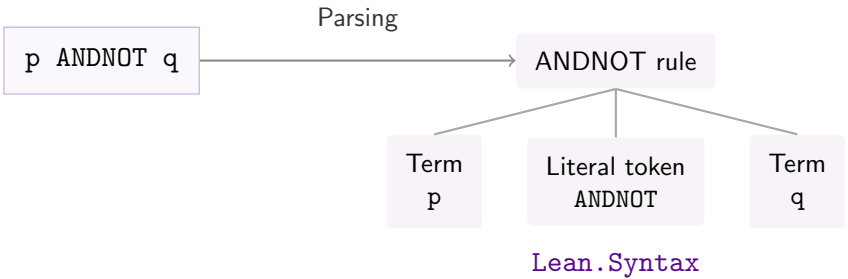
```
inductive Syntax where
  | missing : Syntax
  | node : SyntaxNodeKind → Array Syntax → Syntax
  | atom : String → Syntax
  | ident : Name → Syntax
```

`missing`	`node`	`atom`	`ident`
Parser recovery	Rule kind + children	`ANDNOT`	`p`, `q`

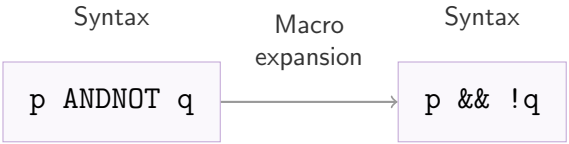
`SyntaxNodeKind` is an alias for `Name`.

Names include `p`, `q`, and `Nat.add`.

Parsing builds a tree

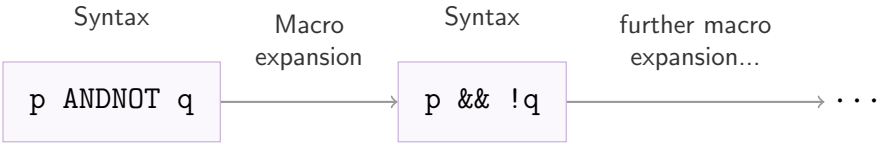


Macros transform syntax



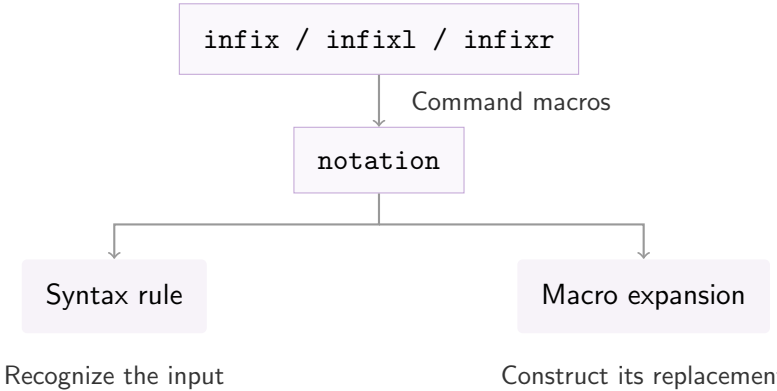
A macro replaces a syntax tree with another syntax tree.

Macros transform syntax



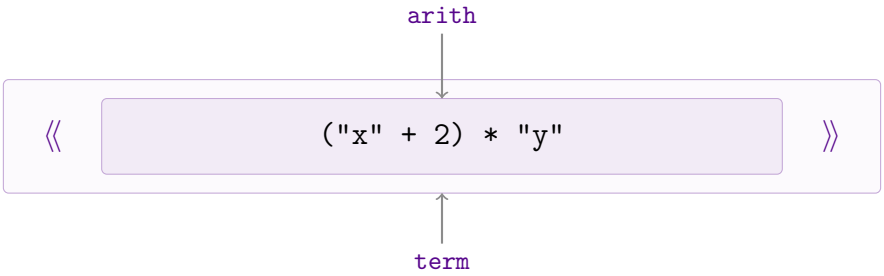
A macro replaces a syntax tree with another syntax tree.

Notation packages syntax and expansion



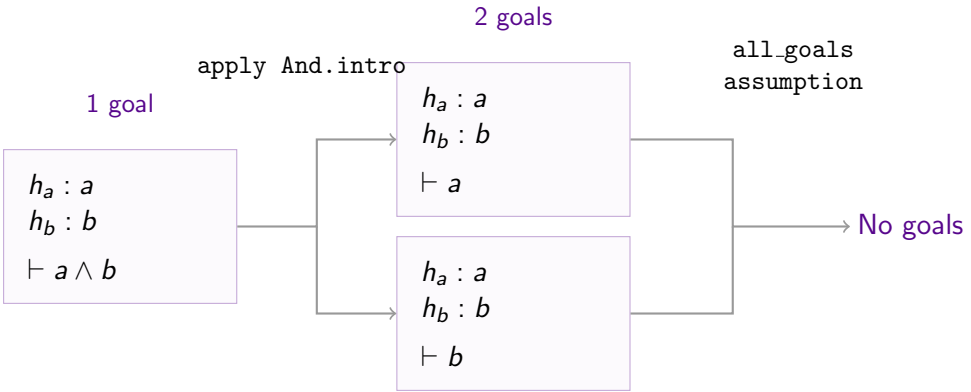
A language of our own

A syntax category can describe a small language.



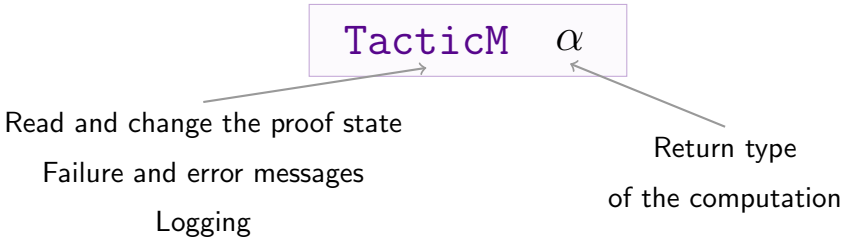
Delimiters embed it into Lean terms.

Tactics and proof states



Proof term: $\langle h_a, h_b \rangle : a \wedge b$

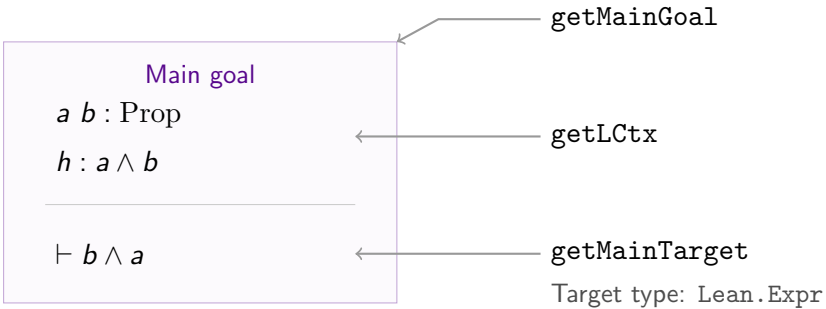
TacticM



Tactics typically have type `TacticM Unit`.

`run_tac` runs a computation in the current proof state.

Reading the main goal



`withMainContext` selects the main goal's context.